COMP8620

Advanced Topics in AI: Intelligent Robotics

Assignment 1 Report

Semester 2, 2026

Student	Rui Luo
UID	u8076655

The Australian National University

Generative AI Disclosure

Two generative-AI tools were used. Claude (Anthropic, accessed via Claude Code) was used throughout: interpreting the assignment, drafting and revising Parts A and B of this report, implementing `planner.py` and `run_planner.py`, designing and running the B4/B5 experiments, and generating the report's figures and appendices. OpenAI Codex was separately consulted for independent review of the assignment requirements, Parts A and B, the B3 implementation, the experiments, and report drafts. All prompts are reproduced verbatim in their original language in `AI_prompts.txt`, included in the submitted source archive.

Part A

Question 1(a): Visibility graph (5 points)

No.
The standard visibility graph requires polygonal forbidden regions and uses their vertices as graph nodes:

$$\mathcal{C}_{\text{obs}} \text{ polygonal} \quad \Longrightarrow \quad \text{shortest-path bends occur at obstacle vertices.}$$

For the sphere robot,

$$k > 0 \quad \Longrightarrow \quad \text{non-point robot} \quad \Longrightarrow \quad \text{construct the problem in configuration space.}$$

Since it only translates on a fixed plane,

$$q = (x, y), \qquad \mathcal{C} \subseteq \mathbb{R}^2.$$

For a polyhedral obstacle O,

$$\mathcal{C}_{\text{obs}} = \{q \in \mathcal{C} \mid R(q) \cap O \neq \varnothing\}.$$

Let B_k^3 be the 3D ball of radius k and Π the plane on which the sphere centre moves. Identifying Π with $\mathbb{R}^2$,

$$\mathcal{C}_{\text{obs}} = \left(O \oplus B_k^3\right) \cap \Pi.$$

Moreover,

$$\begin{aligned} O \text{ polyhedral, } k > 0 \quad &\Longrightarrow \quad O \oplus B_k^3 \text{ has rounded edge and vertex regions,} \\ &\Longrightarrow \quad \left(O \oplus B_k^3\right) \cap \Pi \text{ generally contains curved arcs,} \\ &\Longrightarrow \quad \mathcal{C}_{\text{obs}} \text{ is generally non-polygonal.} \end{aligned}$$

Therefore,

$$\begin{aligned} \mathcal{C}_{\text{obs}} \text{ non-polygonal} \quad &\Longrightarrow \quad \text{the polygonal-forbidden-region assumption is violated,} \\ &\Longrightarrow \quad \boxed{\text{the standard visibility graph cannot be applied directly.}} \end{aligned}$$

Question 1(b): Uniform grid discretisation (5 points)

Yes.
Uniform-grid discretisation requires only a free/occupied test for each cell; it does not require an explicit polygonal representation of $\mathcal{C}_{\text{obs}}$. For the given sphere and obstacles, define

$$\text{FREE-CELL}(c) \iff \forall q \in c,\ R_k(q) \cap Obs = \varnothing \iff c \cap \mathcal{C}_{\text{obs}} = \varnothing.$$

Thus, FREE-CELL directly collision-checks the radius-k sphere; it does not require explicitly constructing the curved boundary of $\mathcal{C}_{\text{obs}}$.

Listing 1: Uniform-grid planner for the translating sphere robot

```
PROCEDURE UNIFORM-GRID(I, G, h)

INPUT: initial configuration I, goal configuration G,
      and cell width h
OUTPUT: a collision-free path, or FAILURE

// C is the 2-D configuration space of the sphere centre.
// GRID(C, h) is the set of square cells of width h.
// c is one cell; c_i and c_j are edge-sharing cells.
// k is the sphere radius; Obs is the workspace obstacle set.
// FREE-CELL(c) tests the entire cell using the given k and Obs.

    V <- empty set // free cells used as graph nodes
    E <- empty set // edges between neighboring free cells

    FOR EACH cell c in GRID(C, h):
        IF FREE-CELL(c):
            V <- V union {c}

    FOR EACH pair c_i, c_j in V that share an edge:
        E <- E union {(c_i, c_j)}

    Gamma <- (V, E) // the resulting grid graph
    c_I <- CELL-CONTAINING(I) // cell containing the initial state
    c_G <- CELL-CONTAINING(G) // cell containing the goal state

    IF c_I is not in V or c_G is not in V:
        RETURN FAILURE

    // P is an ordered sequence of cells from c_I to c_G.
    P <- GRAPH-SEARCH(Gamma, c_I, c_G)

    IF P does not exist:
        RETURN FAILURE

    // CENTRES(P) gives their centres in path order.
    // POLYLINE joins I, these centres, and G by line segments.
    RETURN POLYLINE(I, CENTRES(P), G)
```

Since every retained cell is obstacle-free, straight-line segments connecting the centres of adjacent retained cells are collision-free. However, the planner may miss narrow passages when h is too large, so its success depends on the grid resolution.

Question 2(a): Upper bound on workspace path length (15 points)

Let

$$\Delta\theta_i = \theta_i' - \theta_i, \qquad \delta = d_C(q, q') = \max_{1 \le i \le K} |\Delta\theta_i|.$$

Then

$$|\Delta\theta_i| \le \delta, \qquad i = 1, \ldots, K.$$

Parameterise the straight configuration-space path by

$$q(s) = q + s(q' - q), \qquad s \in [0, 1],$$

so that $\dot{\theta}_i(s) = \Delta\theta_i$.
Consider an arbitrary material point P on link m, where $1 \le m \le K$. Let $r_i(P, s)$ be its distance from joint i at configuration $q(s)$. For each upstream joint $i \le m$,

$$r_i(P, s) \le (m - i + 1)L.$$

Changing θ_i rotates the subchain from joint i onward about that joint. Along the path, an infinitesimal increase ds changes joint i by

$$|d\theta_i| = |\Delta\theta_i|\, ds.$$

If only joint i is considered, P moves along a circular arc centred at joint i with radius $r_i(P, s)$. Its infinitesimal path-length contribution is therefore

$$d\ell_i(P, s) = r_i(P, s)|\Delta\theta_i|\, ds.$$

All upstream joints move simultaneously. By the triangle inequality, the actual infinitesimal displacement of P is no greater than the sum of these contributions. Hence,

$$\begin{aligned}
\ell(P) &\le \int_0^1 \sum_{i=1}^{m} r_i(P, s)|\Delta\theta_i|\, ds \\
&\le L\delta \sum_{i=1}^{m} (m - i + 1) \\
&= \frac{Lm(m+1)}{2}\, \delta \\
&\le \frac{LK(K+1)}{2}\, \delta.
\end{aligned}$$

Substituting $\delta = d_C(q, q')$ gives, for every point P on the robot,

$$\boxed{\ell(P) \le \frac{LK(K+1)}{2}\, d_C(q, q')}.$$

Thus, a valid upper bound is

$$\boxed{\alpha = \frac{LK(K+1)}{2}}.$$

Question 2(b): Collision-free neighbourhood radius (8 points)

For any

$$q' \in \mathrm{Neigh}(q), \qquad d_C(q, q') \le \tau,$$

parameterise the straight configuration-space path as

$$q(s) = q + s(q' - q), \qquad s \in [0, 1].$$

Since $s \in [0, 1]$, substituting $q(s)$ into the definition of d_C gives

$$\begin{aligned} d_C(q, q(s)) &= \max_{1 \le i \le K} \left| [\theta_i + s(\theta_i{}' - \theta_i)] - \theta_i \right| \\ &= \max_{1 \le i \le K} s|\theta_i{}' - \theta_i| \\ &= s \max_{1 \le i \le K} |\theta_i{}' - \theta_i| \\ &= s\, d_C(q, q') \\ &\le s\tau \le \tau, \end{aligned}$$

By Part 2(a), every point on the robot travels a workspace path of length at most $\alpha d_C(q, q(s)) \le \alpha\tau$, so its displacement is at most $\alpha\tau$. Moving every robot point by at most $\alpha\tau$ can reduce the minimum robot–obstacle distance by at most $\alpha\tau$. Hence, by the triangle inequality,

$$d_W(q(s), Obs) \ge d_W(q, Obs) - \alpha\tau.$$

The right-hand side is the initial clearance minus the maximum possible approach towards the obstacles. To guarantee collision-free motion, this lower bound must be positive:

$$d_W(q, Obs) - \alpha\tau > 0 \implies \alpha\tau < d_W(q, Obs) \implies \tau < \frac{d_W(q, Obs)}{\alpha},$$

where $\alpha > 0$. Under this condition, $d_W(q(s), Obs) > 0$ for every $s \in [0, 1]$, so the entire path from q to q' is collision-free.
Since $\mathrm{Neigh}(q)$ includes its boundary and obstacle contact ($d_W = 0$) is a collision, equality cannot guarantee safety. Therefore,

$$\boxed{0 \le \tau < \frac{d_W(q, Obs)}{\alpha}}$$

The supremum of the guaranteed radii is

$$\boxed{\tau_{\sup} = \frac{d_W(q, Obs)}{\alpha} = \frac{2\, d_W(q, Obs)}{LK(K+1)}},$$

where $\alpha = LK(K+1)/2$ from Part 2(a).

Part B

Question 1: Centralised configuration space (2 points)

For $K \in \{1, \ldots, 5\}$, let $\mathcal{W}$ be the bounded physical workspace and

$$\mathcal{W}_r = \{p \in \mathcal{W} \mid B_r(p) \subseteq \mathcal{W}\}$$

be the admissible region for a drone centre. If

$$p_i = (x_i, y_i, z_i) \in \mathbb{R}^3$$

is the centre of drone i, centralised planning treats the K drones as one composite robot, so a joint configuration is

$$q = (p_1, \ldots, p_K) \in \mathcal{C} = \mathcal{W}_r^K \subseteq \mathbb{R}^{3K}, \qquad \boxed{\dim(\mathcal{C}) = 3K}.$$

Let $B_r(p_i)$ be drone i's bounding sphere and let Obs be the union of the workspace obstacles. The collision-free subset is

$$\mathcal{C}_{\text{free}} = \left\{ q \in \mathcal{C} \;\middle|\; \begin{array}{ll} B_r(p_i) \cap Obs = \varnothing, & i = 1, \ldots, K, \\ \|p_i - p_j\|_2 > 2r, & 1 \le i < j \le K \end{array} \right\}.$$

Question 2: Sampling-based planner design (6 points)

I use bidirectional RRT-Connect [3] for three reasons:

(1) This is a single-query problem in a continuous $3K$-dimensional configuration space, with validity checks available from the simulator.
(2) Growing trees from both endpoints and greedily connecting them focuses computation on finding a feasible path within the 20-second limit.
(3) PRM's reusable roadmap is unnecessary for one query, while RRT* spends additional computation on path optimisation that is not required here.

For $q = (p_1, \ldots, p_K)$ and $q' = (p'_1, \ldots, p'_K)$, use

$$d(q, q') = \max_{1 \le i \le K} \|p_i - p'_i\|_2, \qquad \text{Steer}(q, q') = q + \min\left\{1, \frac{\eta}{d(q, q')}\right\}(q' - q),$$

where $\eta > 0$ is the extension length, set to 5% of the workspace diagonal so it scales with the problem instance rather than being a fixed constant. SAMPLE-FREE samples uniformly within the bounds and rejects states failing `sim.is_valid`; NEAREST minimises d by brute-force linear scan over the tree, which is appropriate here because a single `motion_valid` call (interpolated at fixed step size) costs far more than comparing d against the handful-to-low-hundreds of nodes a 20-second budget produces at $K \le 5$, so a spatial index would not pay for itself. An edge is accepted only by `sim.motion_valid`. The goal-tree root q_G satisfies both `sim.is_valid` and `sim.is_goal`.

Listing 2: Centralised RRT-Connect

```
PROCEDURE RRT-CONNECT(sim, eta, T_max)

INPUT: simulator sim, extension length eta, and time limit T_max
OUTPUT: a collision-free joint path, or FAILURE

// q_I is the initial configuration; q_G is a valid configuration
// satisfying the goal condition.
// Each tree node stores a joint configuration and a parent pointer.
// T_I and T_G are rooted at q_I and q_G, respectively.
```

```
// T_a is the active tree; T_b is the opposite tree.
// eta is the maximum length of one extension.

    deadline <- CLOCK() + T_max
    q_I <- sim.initial_configuration
    q_G <- VALID-GOAL-SAMPLE(sim, deadline)

    IF q_G does not exist or NOT sim.is_valid(q_I):
        RETURN FAILURE

    IF sim.motion_valid(q_I, q_G):
        RETURN [q_I, q_G]

    T_I, T_G <- trees rooted at q_I and q_G
    T_a, T_b <- T_I, T_G

    WHILE CLOCK() < deadline:
        // q_rand is a randomly sampled collision-free configuration.
        q_rand <- SAMPLE-FREE(sim, deadline)

        IF q_rand does not exist:
            RETURN FAILURE

        // q_new is the node returned by extending the active tree.
        status, q_new <- EXTEND(T_a, q_rand, eta, sim)

        IF status != TRAPPED:
            // q_meet is the node reached by the opposite tree.
            status_b, q_meet <- CONNECT(T_b, q_new, eta, sim, deadline)

            IF status_b == REACHED:
                // T_a identifies which parent chain contains q_new.
                RETURN JOIN-PATHS(T_I, T_G, q_new, q_meet, T_a)

        T_a, T_b <- T_b, T_a

    RETURN FAILURE

// EXTEND takes one collision-free step from T toward q_target.
PROCEDURE EXTEND(T, q_target, eta, sim)
    q_near <- NEAREST(T, q_target, d)

    IF d(q_near, q_target) == 0:
        RETURN REACHED, q_near

    q_new <- STEER(q_near, q_target, eta)

    IF NOT sim.motion_valid(q_near, q_new):
        RETURN TRAPPED, q_near

    add q_new to T with parent q_near

    IF q_new == q_target:
        RETURN REACHED, q_new

    RETURN ADVANCED, q_new

// CONNECT repeatedly extends T until it is trapped or reaches q_target.
```

```
PROCEDURE CONNECT(T, q_target, eta, sim, deadline)
    q_last <- NEAREST(T, q_target, d)

    WHILE CLOCK() < deadline:
        status, q_last <- EXTEND(T, q_target, eta, sim)

        IF status != ADVANCED:
            RETURN status, q_last

    RETURN TIMEOUT, q_last

// JOIN-PATHS splices the branch ending at q_new (grown in T_a) with the
// branch ending at q_meet (grown in T_b) into one q_I -> q_G path.
PROCEDURE JOIN-PATHS(T_I, T_G, q_new, q_meet, T_a)
    branch_new <- trace q_new back to its tree's root, root-to-node order
    branch_meet <- trace q_meet back to its tree's root, root-to-node order

    IF T_a == T_I:
        RETURN branch_new followed by reverse(branch_meet)
    ELSE:
        RETURN branch_meet followed by reverse(branch_new)

// Return a collision-free joint configuration inside the goal regions.
PROCEDURE VALID-GOAL-SAMPLE(sim, deadline)
    q_G <- the vector of goal centres

    IF sim.is_valid(q_G) AND sim.is_goal(q_G):
        RETURN q_G

    WHILE CLOCK() < deadline:
        q_G <- sample one point inside each drone's goal region

        IF sim.is_valid(q_G) AND sim.is_goal(q_G):
            RETURN q_G

    RETURN FAILURE
```

All tree edges pass `motion_valid`. JOIN-PATHS uses T_a to orient their parent chains into a collision-free path from q_I to the valid goal q_G; a finite deadline may return FAILURE.

Question 3: Implementation and demonstration (15 points)

The submitted `planner.py` implements Algorithm 2 with the supplied MultiDrone collision and local-motion primitives [2]. `run_planner.py` loads a simulator-format YAML file, infers K, sets the seed and scaled η, and enforces the 20-second total demo budget.

Reproduction commands.

```
# Complete template; arguments in brackets are optional.
python3 run_planner.py --environment <scenario.yaml> \
  [--num-drones K] [--time-limit SECONDS] [--seed SEED] [--eta ETA] \
  [--bridge-prob P] [--bridge-sigma SIGMA] [--local-bridge] [--visualize]
# --num-drones inferred from the YAML if omitted
# --time-limit demo budget in seconds (default 20)
# --seed RNG seed (default 0)
# --eta max extension length (default 5% of the workspace diagonal)
# --bridge-prob bridge-test sampling probability (default 0 = B2 baseline)
# --bridge-sigma bridge-test perturbation stdev (default eta)
# --local-bridge anchor bridge attempts near the active tree (EST-style)
```

```
# --visualize open the simulator's 3D viewer after a success

# Minimal template.
python3 run_planner.py --environment <scenario.yaml> --visualize

# Example.
python3 run_planner.py --environment environments/E1_sparse.yaml --visualize
```

By default, the runner infers K, uses a 20-second limit, seed 0, scaled η, and baseline uniform sampling.

Path validation. Before reporting success, the runner independently rechecks the initial configuration, every consecutive edge with `sim.motion_valid`, and the final configuration with `sim.is_goal`.

E12 checks goal-region sampling when the goal centres themselves are invalid; all 25/25 runs succeeded in 0.06 ± 0.01 s. E13 seals the initial cluster inside a closed box; all 10/10 runs returned FAILURE without a hang or invalid path, with capped planner runtime 20.01 ± 0.00 s excluding simulator setup.

Question 4: Environment-complexity experiment (6 points)

Research question and hypothesis

The experiment asks how obstacle density and free-space structure affect baseline RRT-Connect for five drones. Four hypotheses are tested:

(1) Increasing obstacle density increases planning time.
(2) Narrow or sequential passages reduce success, because fewer random samples can connect the two regions.
(3) Multiple alternative passages of the same width improve success relative to a single passage.
(4) Compound constraints and local traps reduce finite-time success.

Experimental design

Layouts for every environment referenced below are collected in Appendix A.

Table 1: Controlled environment-complexity comparisons.

Comparison	Environments	Varied feature	Trials each
Obstacle density	E0–E3	$0, 3, 7, 14$ obstacles; no critical bottleneck	25
Passage width	E4, E5	One gap of width 14 or 2	25
Route redundancy	E5, E6	One or three width-2 gaps	25
Offset passages	E7	Two sequential width-2 gaps	15
Stress tests	E9–E11	2-D aperture, coupled swap, or dead-end trap	25

Protocol.

- **Drones and goals:** $K = 5$, the same start and goal configurations used across every environment.
- **Workspace:** $[0, 50]^3$, drone radius 0.3.
- **Planner parameters:** $\eta = 0.05\sqrt{3(50)^2} = 4.330$ (5% of the workspace diagonal), uniform sampling only (no bridge sampling), 20-second timeout per run.
- **Seeds:** 0–24 for E0–E6 and E9–E11; 0–14 for E7.

- **Execution:** runs were sequential; timing used `time.perf_counter` around the planner call only, excluding simulator construction.
- **Hardware/software:** 2.4 GHz eight-core Intel Core i9 MacBook Pro, 16 GB RAM, macOS 26.5.2, NumPy, the supplied MultiDrone simulator.

Metrics and confidence intervals. Success means returning a validated path within 20 seconds. Capped runtime is the elapsed planner time, with failed runs retaining their observed timeout duration (approximately 20 seconds). For n runs with sample standard deviation s, the two-sided Student-t interval is

$$\bar{t} \pm t_{0.975,n-1}\frac{s}{\sqrt{n}}.$$

Explicit waypoint witnesses for E9 and E11 were separately validated state-by-state and edge-by-edge with `is_valid` and `motion_valid`. Thus, their zero baseline successes indicate finite-time planner difficulty rather than infeasibility.

Results

Table 2: Baseline RRT-Connect performance; capped runtime is mean ± 95% Student-t CI.

Environment	Structure	n	Success	Capped runtime (s)
E0	Empty	25	25/25	0.04 ± 0.00
E1	Sparse	25	25/25	0.12 ± 0.01
E2	Moderate	25	25/25	0.32 ± 0.07
E3	Dense	25	25/25	1.03 ± 0.19
E4	Wide passage	25	25/25	0.16 ± 0.03
E5	Narrow passage	25	20/25	10.19 ± 2.67
E6	Three narrow passages	25	25/25	2.95 ± 0.50
E7	Offset sequential passages	15	1/15	19.38 ± 1.35
E9	Coupled y-z aperture	25	0/25	20.00 ± 0.00
E10	Bottleneck + drone swap	25	22/25	7.97 ± 2.38
E11	Dead-end trap	25	0/25	20.00 ± 0.00

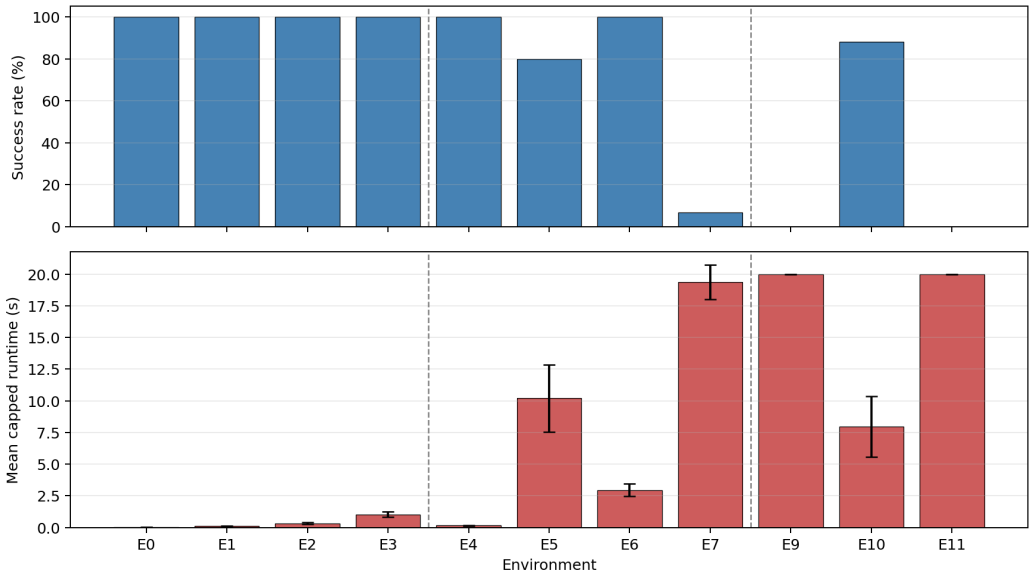


Figure 1: Success rate and mean capped runtime for the baseline planner. Dashed lines separate density (E0–E3), passage-structure (E4–E7), and additional stress tests (E9–E11). Failed runs retain their timeout duration; error bars show 95% Student-t confidence intervals.

Discussion

(1) **Obstacle density (E0–E3).** 100% success throughout; mean runtime rose from 0.04 to 1.03 s. Consistent with more collision rejection and local-motion work, while broad alternative routes preserved free-space coverage.

(2) **Passage geometry vs. obstacle count (E4–E7).** Geometry matters more than raw count: narrowing E4's single gap from 14 to 2 units (E5) dropped success from 100% to 80% and raised runtime from 0.16 to 10.19 s; three width-2 alternatives (E6) restored 100% success; two offset transitions (E7) succeeded only once in 15 runs. Paths for E5–E7 are in Appendix B.

(3) **Additional axes (E9–E11).** E9's two-dimensional aperture and E11's dead-end trap both produced 0/25 baseline successes, although the verified witnesses establish that both are feasible. E10 combines E5's bottleneck with a full drone-order reversal; its 22/25 success rate indicates that the bottleneck remained the dominant difficulty. Layouts are in Appendix A.

(4) **Limitations.** Results use one start–goal set, $K = 5$, the selected parameters, and a 20-second cutoff; E0–E3 vary obstacle geometry rather than forming perfectly nested scenes, and E7 has fewer trials. The results are comparative, not universal.

Question 5: Scalability with the number of drones (4 points)

Increasing K affects the planner in four distinct ways:

(1) **Dimension.** For K translating drones, $q \in \mathbb{R}^{3K}$, so the configuration-space dimension grows linearly as $3K$.

(2) **Collision-check cost.** A single state check requires K drone–obstacle checks and $\binom{K}{2} = O(K^2)$ inter-drone checks; MOTION-VALID repeats these along each local path.

(3) **Nearest-neighbour cost.** With a brute-force tree of N nodes, both the nearest-neighbour query and node storage cost $O(NK)$.

(4) **Sample efficiency.** Increasing K reduces the fraction of jointly valid samples and the relative measure of coordinated passages in the joint space.

To isolate the effect of K, E8 contains no static obstacles but reverses the drones' initial order, making the direct joint-space segment invalid. Each condition used the baseline planner and 25 seeds. Layouts and successful paths for $K = 2, 3, 4, 5$ are in Appendices A and B.

Table 3: Scalability in E8; runtime is mean ± 95% Student-t CI over 25 runs.

K	Dimension	Pair constraints	Success	Runtime (ms)
2	6	1	25/25	21.72 ± 0.61
3	9	3	25/25	34.84 ± 1.40
4	12	6	25/25	44.68 ± 4.27
5	15	10	25/25	54.32 ± 3.79

(1) **Result.** All tested E8 runs ($K = 2, \ldots, 5$) succeeded; mean runtime increased by a factor of 2.5 from $K = 2$ to $K = 5$, consistent with the growing state dimension and pairwise checks.
(2) **Scope.** E8 is obstacle-free, so it does not measure the stronger dimensional effect of cluttered narrow passages, already visible at $K = 5$ in E5 and E7.
(3) **Conclusion.** The $K = 1$ case has dimension 3 and no inter-drone pair constraints; E8 starts at $K = 2$ because a swap requires two drones. Across the required $K \in [1, 5]$ range, the implementation is practical in open scenes; in clutter, the dominant limitation is repeated local collision checking together with the low probability of sampling useful coordinated configurations.

References

[1] H. Kurniawati, *COMP8620 Advanced Topics in AI: Intelligent Robotics, Weeks 1–3 lecture materials*, Australian National University, 2026.

[2] RDLLab, "MultiDrone Simulator," 2026. Available: https://github.com/RDLLab/MultiDrone. Accessed: 26 August 2026.

[3] J. J. Kuffner, Jr. and S. M. LaValle, "RRT-Connect: An efficient approach to single-query path planning," in *Proceedings of the 2000 IEEE International Conference on Robotics and Automation*, vol. 2, 2000, pp. 995–1001.

A Environment Layouts

Dots denote initial drone positions; translucent circles denote the corresponding goal regions.

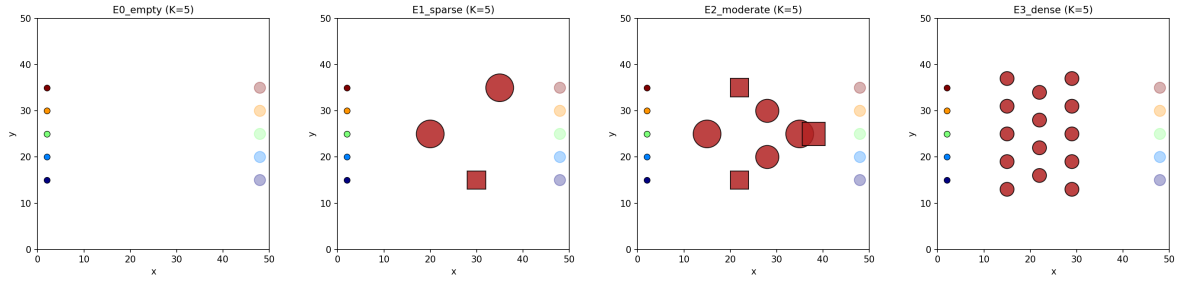


Figure 2: E0–E3: obstacle-density comparison (0, 3, 7, 14 obstacles).

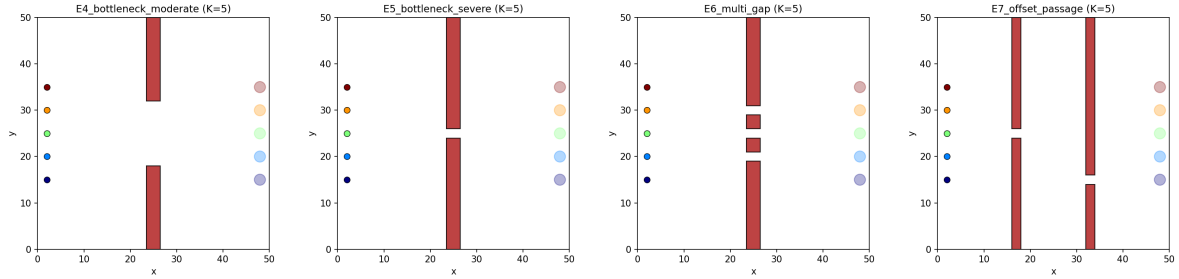


Figure 3: E4–E7: passage width, route redundancy, and offset passages.

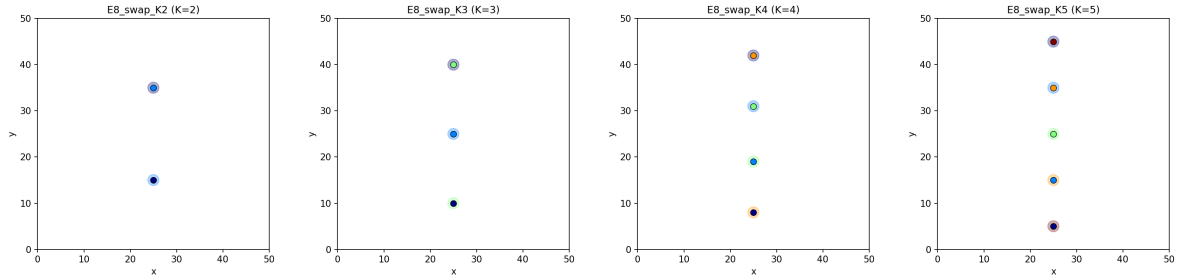


Figure 4: E8: drone-swap scalability series, $K = 2, 3, 4, 5$ (Question 5).

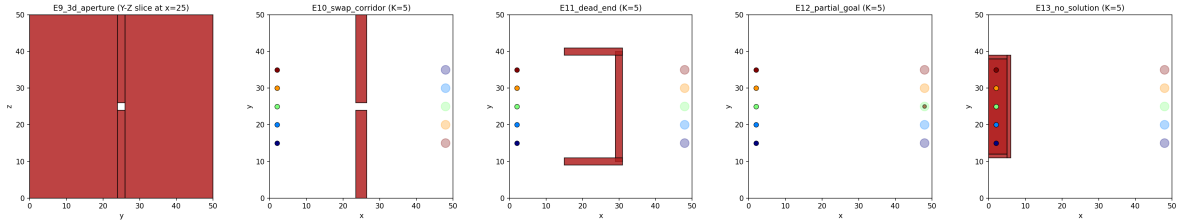


Figure 5: E9–E13: additional environments (E9–E11 in Table 2; E12–E13 are the Question 3 path-validation checks). E9 is shown as its y-z cross-section, the rest top-down.

B Successful Paths

One successful path per environment. E9, E11, E13 omitted (0% success, Table 2).

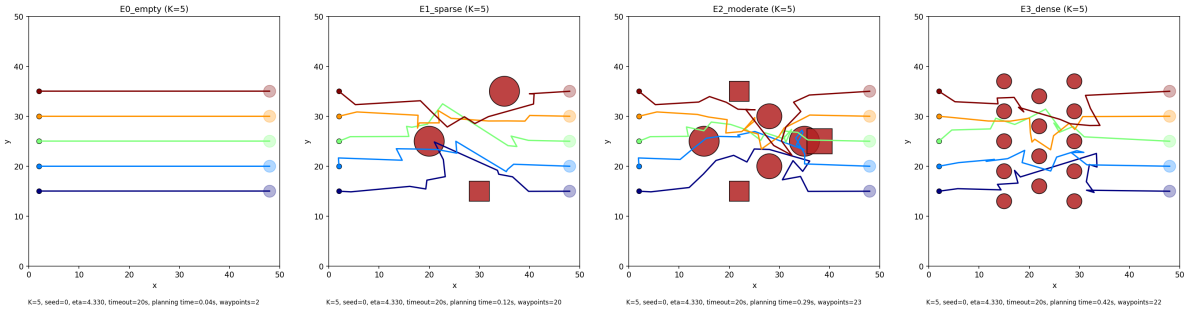


Figure 6: Successful paths, E0–E3.

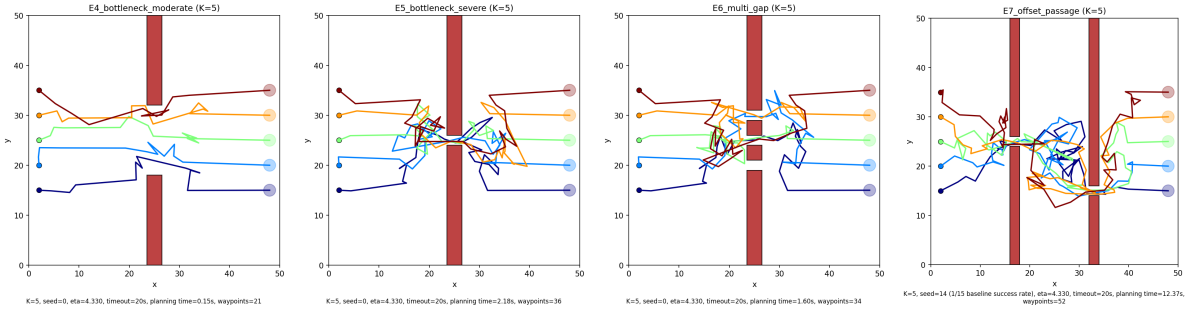


Figure 7: Successful paths, E4–E7. E7 shows its one success out of 15 seeds.

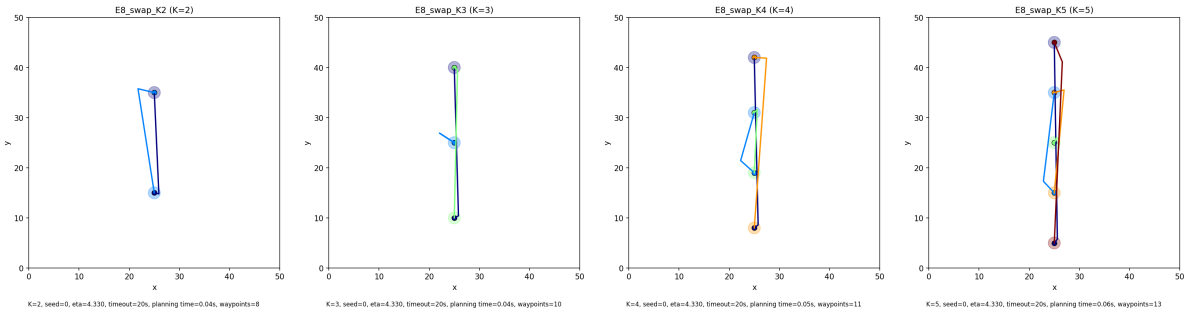


Figure 8: Successful paths, E8 drone-swap series ($K = 2, 3, 4, 5$).

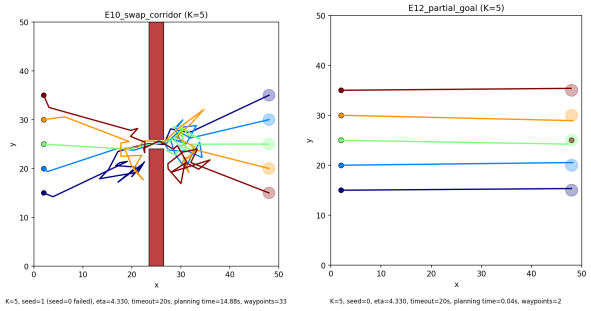


Figure 9: Successful paths, E10 (bottleneck + drone swap) and E12 (partial-goal fallback).